Flux

A Decentralized Cloud for Autonomous Compute

A short introduction

Tadeas Kmenta

September 7, 2026

What Flux is, as measured on 2026-09-03; how it works; what is changing in v9 and when; what it is for; and what does not work yet. Every number here is taken from a longer technical paper that cites source code for each claim about deployed behaviour, and nothing here goes beyond it.

Where this document describes something planned rather than running, the sentence says so. Where a claim has a limit, the limit sits beside it in a grey box. That is the method, and it is the reason the document can be trusted.

Companion to *Flux v9: A Decentralized Cloud for Autonomous Compute*.

In one page

What it is. A decentralized cloud of 6,489 independently operated, collateralised machines running 1,195 deployed applications, coordinated by its own blockchain. An operator locks FLUX to join; the chain decides who may host and who is paid. Nobody signs a contract with anyone.

What already runs on it. Not a proposal. By census: web services, blockchain infrastructure, and 27 scientific-simulation deployments doing real protein-folding work on about 9 % of the fleet's committed CPU. A general-purpose cloud with science on it, into which AI is one arriving workload among several.

The bet. The next tenant is software, not a person. Every major cloud is built for a human with a card and an account. Flux is being rebuilt for a caller that is a program — one that provisions, pays, and releases capacity with no human anywhere. It can, for one narrow and decisive reason: on Flux, paying is a signed transaction, not a billing relationship. *A program can hold a key. It cannot hold a corporate account.*

Why this fleet. The thing that centralises frontier AI — sharding one model across many accelerators that must be wired together — simply does not apply to a model that fits on one device. And the models most people actually need fit on one device. A dispersed fleet with a great deal of memory is a structurally sensible home for them.

What changes. In July 2027, at block 3,787,502, operators stop being paid purely by new issuance: a fifth of every application payment flows into a pool that pays them instead. Ten parallel-asset chains become two. The shielded pool is retired. A collateral-weighted vote can remove an application. Each has a date, and each is planned rather than shipped until it lands.

What does not work yet. Hosting an application earns a node nothing extra, by design — so the whole hosting guarantee rests on an eligibility gate that today is bound to neither the operator nor the machine. Four owner identities together hold enough weight to remove any application. There will be no chain-level privacy. One Foundation-run API is the sole source of where applications live. The bridge is administered by four people from one organisation. None of this is hidden in the long paper, and none is hidden here.

Contents

1 What Flux is

Concrete before abstract. Everything in this section was measured on 2026-09-03 from the network's own public interfaces, and can be measured again by anyone who runs the scripts in the repository.

1.1 A cloud made of collateralised machines

Flux is a decentralized cloud: a network of independently operated machines that run other people's applications, coordinated by a blockchain rather than by a company. On the day of measurement it consisted of 6,489 **nodes** running 1,195 **deployed applications**.

What makes a machine a node is not an agreement with anyone. It is a lock. An operator locks a fixed amount of FLUX as collateral, runs the node software, and from then on the chain itself decides whether that machine is eligible to host and to be paid. Leave, and the collateral unlocks. Misbehave in ways the chain can see, and it does not.

Three tiers, three collateral sizes, three hardware floors:

Tier	Collateral	Nodes today	Share of voting weight
Cumulus	1,000 FLUX	3,247	3.67 %
Nimbus	12,500 FLUX	1,615	22.81 %
Stratus	40,000 FLUX	1,627	73.52 %
Total locked		6,489	88,514,500 FLUX

Two things in that table are worth noticing before moving on. First, the collateral is real and large: 88,514,500 FLUX is locked across the fleet, about a fifth of everything ever issued. Second, weight follows collateral, not headcount — half the nodes are Cumulus, but they carry under four percent of the weight. That fact returns in Section 6, where it stops being a curiosity.

1.2 A chain that pays for capacity, not for hashing

The network runs on its own chain. Until October 2025 that chain was proof-of-work; at block 2,020,000 it switched to *Proof of Node*, in which the right to produce a block is drawn by lot from the collateralised nodes rather than won by burning electricity. Blocks arrive every thirty seconds. Node income is a position in a rotation, not a lottery ticket.

The consequence for a reader is simple: the thing the chain rewards is *being available with collateral at stake*, which is exactly the thing a cloud needs its machines to be.

1.3 What actually runs on it

Applications are containers, described by a specification the owner publishes to the chain and pays for by the block. Three-quarters of deployed applications (873 of 1,195) are on the current specification version, and the mix is more general than a newcomer might guess: ordinary web services and content sites; blockchain infrastructure, including full nodes for other chains; and a substantial block of scientific-computing work — 27 protein-folding and structural-biology deployments that between them hold roughly 9 % of the fleet's committed CPU.

That last point matters enough to come back to in Section 4. For now: this is a running, general-purpose cloud, not a proposal for one.

What this does not do yet. Capacity is measurable; utilisation is not. The interfaces this section was measured from report what each node *has* and what is *deployed*, not how busy anything is. The paper states the counts it can stand behind and does not estimate the number it cannot.

2 How it works

One pass through the three layers, with no formalism. The full paper specifies each of these to the depth at which it could be reimplemented; this section aims only at the depth at which it can be understood.

2.1 The chain: a rotation, not a race

Every collateralised node is in a queue. Producing a block moves a node from the front of the queue to the back, and the block reward goes to whoever produced it, split by tier according to a fixed schedule. There is no difficulty to grind and nothing to mine; a node's expected income is a function of how many nodes are in its tier and how long the rotation takes to come round.

Because every block carries the same fixed weight, the chain does not have a longest-chain rule in the Bitcoin sense. Fork choice is a block-count race resolved by which block was seen first, and reorganisations are capped at forty blocks by consensus. That is a deliberate simplification, and it has a cost the full paper treats at length: with no cumulative-work signal in the header, a lightweight client that verifies without a full node cannot be built on today's format.

2.2 Where new FLUX comes from

Every block mints new FLUX, and today that issuance is the whole of an operator's income. The rate is 14.0 FLUX per block, split across the three tiers and a development fund by fixed shares. It falls by ten percent each October — to 12.6 at block 3,071,200 in 2026, then 11.34, and so on for twenty annual steps — and then *freezes* rather than continuing toward zero: from the twentieth reduction onward the chain issues a constant tail of roughly 1,789,219 FLUX a year. Total issued supply reaches 548,043,625.86 FLUX by 2045 and crosses 560,000,000 around 2052.

Two things follow for a holder. There is no hard cap that issuance approaches and stops at — the tail is perpetual by design. And the emission curve is what v9 is built to *replace* as the source of operator income: at PNR activation the operators' share of new issuance is planned to be cut in half, with revenue from applications taking its place (Section 3).

2.3 The cloud: publish, claim, run

A deployment starts as a specification — the container image, the resources it needs, how many copies should run, how long it is paid for. The owner signs it, pays for it, and broadcasts it. That is the whole act of deploying: there is no console and no account.

Nodes then *claim* it. Every eligible node sees the specification, decides whether it can host it, and if so announces its intention and waits ninety seconds. If nobody else has claimed the same instance in that window, it installs and starts the container. If the announced

count is already met, it stands down. The application ends up running on several nodes at once, chosen by whoever was fast rather than by any scheduler.

This is unusual, and the paper says so plainly. There is no placement algorithm in the sense a cloud engineer would expect — no bin-packing, no affinity, no capacity model. What there is instead is an incentive: nodes want to be full, applications want to be replicated, and the ninety- second collision rule keeps them from tripping over each other. It works at the current scale. What it does not do is give a tenant any say over *where* the application lands.

How you reach it. A running application gets a name under `app.runonflux.io` (and `app2.runonflux.io`), resolved by the network's own DNS and fronted by load balancers that health-check each instance and route around the ones that fail. That layer is what makes an application on several anonymous machines look like one service — and it is operated by the Foundation on a sixteen-host fleet, which Section 6 returns to.

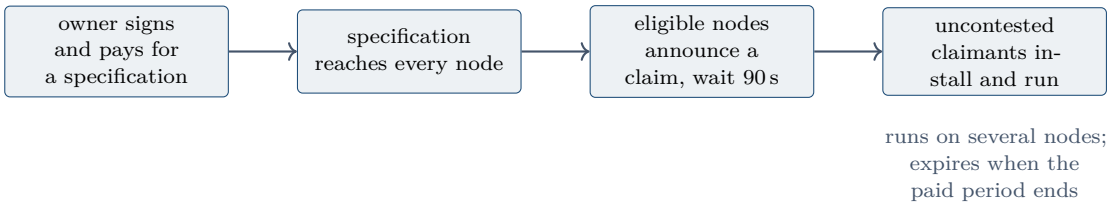

Figure 1: From a signed specification to a running application. No account is opened at any step, and no scheduler decides placement: nodes claim work, and a fixed wait resolves collisions.

2.4 Paying: a transaction, not a relationship

Today, paying for a deployment means one thing: a single signed transaction, for the full price of the period, to an address the chain knows. The price is a function of the resources reserved and the duration — CPU, memory, disk, instances, blocks — and it is a *reservation* price. You pay for what you asked to hold, not for what you used. There is no metering, and therefore nothing to meter, dispute, or attest.

When the paid period ends, the application stops. Renewing it is the owner's act, done the same way. The protocol offers no subscriptions and no automatic renewal; operators who want that convenience can buy it from a third-party service that holds a balance on their behalf, and the paper is explicit that such a service is a business built on Flux, not a feature of it.

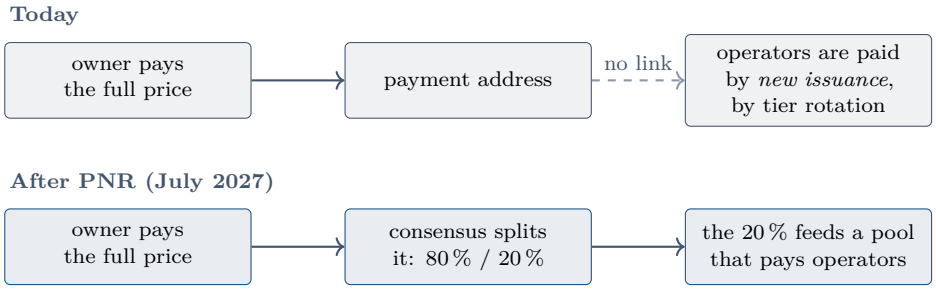

Figure 2: Where a payment goes. Today the money a tenant pays and the money an operator earns are unconnected — operators are paid by issuance. Under PNR, part of every payment flows into a pool that pays operators instead. This is the hinge between the network as it is and as it is being rebuilt.

What this does not do yet. The payment address is a single consensus-known address, and the split is applied by consensus, not by the node software — so a software release cannot change the distribution. What the protocol does *not* give a tenant is any remedy if a node fails to serve what was paid for. Reservation pricing has no delivered-uptime quantity to compute a refund against, and the paper records the decision that any such guarantee lives in the service layer, not the protocol.

3 What changes in v9

Six changes, each with a date. Every one of them is planned rather than shipped unless the sentence says otherwise, and the reader should hold that distinction throughout: this section describes a network being rebuilt, and the full paper marks every element of it that is still undecided.

3.1 Operators are paid by demand, not by issuance

This is the change the others serve. Today an operator's income is new FLUX minted every block; the applications running on the network contribute nothing to it. Under *Progressive Node Rewards* that reverses: consensus takes 20 % of every application payment on arrival and pays it into a pool, and the pool drips into operator rewards over a rolling window of 86,400 blocks — thirty days. The other 80 % goes to the Foundation address that already receives application payments today.

The split is applied by consensus at the address, not by any software a tenant or a node runs. The pool balance is committed in every block and recomputed by every validator, so it cannot be misreported. Activation is at block 3,787,502, expected **1 July 2027**. That height is not arbitrary: it is the block at which the parallel-asset mining allocation is exhausted, computable today from the emission schedule, though no consensus constant encodes it yet; the calendar date follows from the height at the thirty-second block target.

A reader should understand what this does and does not change. It changes *where operator income comes from* — from issuance toward the workloads themselves. It does not, by design, pay a node more for hosting more: every eligible node in a tier draws the same share of the pool regardless of what it runs. The team chose that deliberately, because paying by placement would turn the first-to-claim rule of Section 2 into a race with money on it. The consequence is stated in Section 6.

3.2 Ten parallel assets become two, on one redesigned bridge

Flux exists as a token on ten other chains. That footprint is being consolidated. Ethereum and BNB Chain survive: balances there are carried across to new contracts by snapshot, and holders do nothing. The other eight — Base, Polygon, Avalanche, Ergo, Algorand, Solana, Tron and Kadena — are retired in stages: from March 2027 (block 3,450,000) they become one-directional, with balances redeemable back to the main chain and mining claims still collectable, but nothing swappable into them; the allocation is cut in July, at the block where it is exhausted and PNR activates; and in August (block 3,880,000) the chains shut down, after which unclaimed balances go to the Foundation. Ethereum and BNB Chain move to their new contracts earlier. What the height does not settle is the balance already earned: roughly a quarter of what operators have accrued on these chains has never been claimed, and the forfeiture rule is what closes that.

The numbers were measured rather than assumed, and they are reassuring in a way the raw supply figures are not. Across all ten chains, 223,027,781.21 FLUX is actually held outside the issuer's own addresses. Of that, 97.21 % sits on the two surviving chains and needs no holder action. Only 6,226,900.80 FLUX — 2.79 % — is on the retiring chains and exposed to the window.

The new bridge is mint-and-burn: minting on a parallel chain requires locked backing on the main chain, so the total can never exceed what exists. It is guarded by a 3-of-4 signing quorum, a 1-of-4 emergency pause, a per-chain cap of 250,000,000 FLUX, a rate limit of 10,000,000 FLUX per chain per day, and a 48-hour delay on any upgrade during which a single guardian can cancel it. The quorum is held by four named people from one organisation, and the paper says so.

3.3 A new application specification

Version 9 of the specification adds what the rest of this section needs: a duration expressed in seconds rather than blocks, so lifetimes stop moving when block timing changes; a payment memo that binds a payment to the application it funds, which is what lets PNR attribute revenue; and a machine-type field that lets a specification ask for accelerated hardware. Enforcement is at block 3,050,000, expected in **late October 2026**, with existing applications migrated automatically in the same quarter.

What this does not do yet. The fields are in the schema and not yet in the code that would act on them: as of the survey, the payment memo, referral code and machine-type fields appear nowhere in the placement path. The enforcement height is constrained to follow the implementation, not precede it.

3.4 The network can remove an application

A collateral-weighted vote. Any node operator can file a report against an application on chain; other operators vote by signing a transaction with their collateral key; if votes reach 25 % of the network's weight within a day, the next block carries a ban memo and every node stops the application. The ban is final, the unused prepaid balance is burned, and the deployer may return only as a new application. There is no appeal.

The team's stated position is that this is a network-level veto over content, not merely over behaviour. That is a stronger power than any comparable network exercises, chosen

knowingly, and its principal risk is measured rather than hypothetical — see Section 6.

3.5 Nodes that can prove what they are

Two layers. The first is shipped: ArcaneOS, the operating image nodes run, boots through a measured chain — Secure Boot, a pinned platform key, a hardware root, encrypted storage — and refuses to come up if any link fails. The second is planned: a bonded attestation network in which nodes stake collateral on claims about what they serve, and lose it if a challenger proves them wrong. Its parameters are set; its slashing construction, the paper is candid, is unsolved.

3.6 The shielded pool is retired, and what that costs

Flux's chain inherited Zcash's private-transaction machinery. It has been sealed to new entrants since 2021 and today holds 96,479.94 FLUX — two hundredths of one percent of supply. The decision is to retire it: announced now, sunset at block 3,600,000 in April 2027, before PNR activates, and then remove the code. That makes the chain smaller, removes an inherited trusted setup, and makes supply exactly auditable. It also means **Flux will have no chain-level privacy**: every payment will be publicly linkable, as on Bitcoin. The paper treats payment privacy as an open problem from that point, not a delivered feature.

3.7 And one change the fleet's own design forces

A post-quantum note that is specific to Flux. Because registering a node publishes the collateral's public key on chain, 88,514,500 FLUX — 20.61 % of everything issued — sits in outputs a sufficiently capable quantum computer could spend immediately. Bitcoin's comfort, that an unspent output hides its key, does not apply here. The adopted answer uses the one lever Flux has and Bitcoin does not: node collateral is a registered set that must keep transacting to be paid, so migration to a post-quantum key can be made a condition of eligibility. Operators who do not migrate stop earning; nothing is ever frozen.

4 What it is for

Everything so far has been description. This section is the argument: what a network of this shape is good for, and why that is about to matter more than it did.

4.1 The bet: the next tenant is software

Open any cloud console today and you will find an interface built for a person. Pick a region. Choose an instance size. Enter a card. Agree to terms. Every major cloud is built on the assumption that a human is making the decision, and the whole surface — accounts, billing relationships, credit checks, support tiers — follows from it.

Flux is being rebuilt on the opposite assumption: that the caller is a program. Something that provisions capacity when it needs it, pays for it, scales it, and releases it when the work is done, with no human in the loop and no account anywhere.

The reason this is possible here and awkward everywhere else is narrow and concrete. On Flux, paying for a deployment is a signed transaction. It is not a billing relationship. There is no account to open, no card to hold, no legal entity to be. **A program can hold**

a key. It cannot hold a corporate account. That one asymmetry is the whole of the positioning claim, and everything in this section follows from it.

This is not a prediction that people stop using clouds. It is a claim about where the growth goes. Software that writes software, agents that run errands, pipelines that spin up a hundred short-lived workers and vanish — these are workloads whose natural interface is a key and a transaction, not a console and an invoice.

4.2 One substrate, and it already runs more than AI

It would be easy, and wrong, to present Flux as an AI network.

What actually runs on it today, by census rather than by aspiration, is a general-purpose mix: ordinary web applications, blockchain infrastructure, content sites — and **27 folding and Rosetta deployments** doing real protein-folding and structural-biology work, which alone account for roughly 9 % of committed fleet CPU. Scientific simulation is not a use case Flux is hoping to attract. It is a use case that is already there, paying, at scale.

That matters more than it sounds, and the reason is economic rather than technical. A network that serves only inference is idle whenever inference is idle. A network that serves web hosting, scientific simulation, blockchain infrastructure and model inference — from the same collateral, the same placement, the same payment path — has a demand base whose parts do not move together. The generality is the resilience.

So the claim is not that Flux becomes an AI cloud. It is that Flux is a compute substrate on which AI is one arriving workload class among several that are already running.

4.3 Cloud and edge, converging

FluxCloud and FluxEdge are to merge into a single product, offering accelerated compute through one interface. A tenant should describe the work and let placement decide where it runs — a container on a commodity node, or a GPU on a datacenter machine — without having to know which product they are in. That convergence is planned, not shipped.

What this does not do yet. Convergence is at present an architectural intention with thin implementation behind it. The only concrete artefact is a small, new capability specification, and the placement code that would have to consume it does not read it yet: the three fields the new specification adds for this purpose appear nowhere in the placement path. A merge announced is not a merge shipped, and this document would rather say so than let a reader discover it later.

4.4 Why small models fit this fleet in particular

The interesting question is not whether a decentralized network can run AI workloads. It is which ones, and why.

Frontier-model inference is centralised for a specific engineering reason: the model does not fit on one device, so it is sharded across many accelerators that must exchange activations at every layer. That demands an interconnect — machines physically close, wired together at bandwidths a wide-area network cannot approach. A fleet of commodity machines scattered across the world is genuinely unsuited to it, and no amount of decentralization enthusiasm changes that.

But that requirement does not exist for a model that fits on a single device. A model small enough to sit in one machine's memory needs no interconnect at all, because

there is nothing to shard. The property that makes a dispersed fleet useless for frontier inference is simply absent for the models most people actually need: the one that drafts the email, summarises the document, triages the inbox, answers the question. Most work does not require a frontier model, and the work that does not is precisely the work this shape of network can serve.

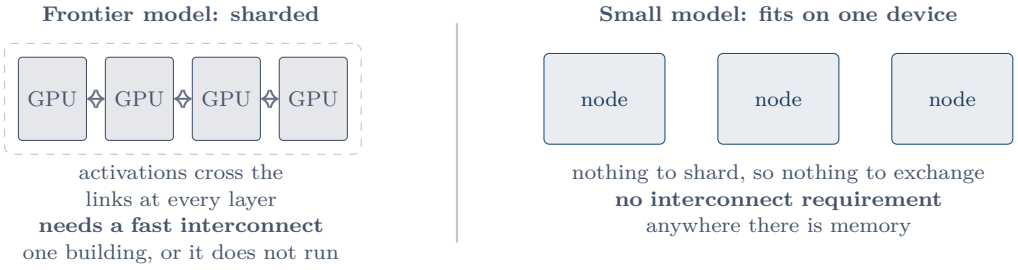

Figure 3: Why the constraint that centralises frontier inference does not bind here. Sharding a model across accelerators forces them into one building; a model that fits on one device imposes no such requirement, and a dispersed fleet stops being a disadvantage.

There is a second, sharper reason the fit is good, and it is a property of this fleet rather than of decentralized networks generally. Decode speed follows a model's *active* parameters, but residency — whether the model fits at all — follows its *total* parameters. Mixture-of-experts models separate the two: they read only a few billion parameters per token while having to hold tens of billions in memory. Such models therefore want **RAM** more than they want memory bandwidth. RAM is what this fleet has, in quantity, across thousands of independent machines.

On commodity dual-channel hardware a four-billion-parameter quantised model runs at roughly 12 tokens per second; on an eight-channel server, roughly 40. Across the network's tenant-available memory the capacity, at full utilisation, is on the order of tens of thousands of concurrent small-model instances.

What this does not do yet. Those rates are a memory-bandwidth model — achievable bandwidth divided by bytes read per token — and have *not* been measured on Flux hardware. A benchmark on each tier's reference machines is scheduled for Q4 2026, and until it lands these are estimates rather than results. The capacity figures are upper bounds at full utilisation, not availability: roughly 9 % of fleet CPU is already committed to the scientific workloads above. And the GPU path has a harder limit — accelerators are attached whole, with no subdivision, so a tenant wanting a fraction of a card cannot have one, and a GPU request under contention is silently dropped rather than refused.

4.5 What an autonomous tenant can and cannot do

The agent story is the most likely place for a reader to assume more than is true, so it is worth closing on precisely.

An agent on Flux can hold a key, sign a deployment specification, pay for it, watch it run and let it expire — with no human, no account, and no intermediary at any step. That is real today for the payment path, and it is the direct consequence of settlement being a transaction rather than a relationship.

What an agent cannot be given is a spending limit the network enforces. Flux inherits Bitcoin's model, in which validating a transaction is a function of that transaction and the coins it spends — and nothing else. A rule like *spend at most five* FLUX *per day* requires knowing what the key spent yesterday, and there is nowhere for that rule to live. **An agent's budget is whatever you fund its key with, and that is the honest bound.** Anything richer needs a co-signer, which puts back the intermediary the design exists to remove.

That is a real constraint and the paper states it rather than working around it. It is also, in practice, a workable one: funding a key with what you are willing to lose is a familiar discipline, and it fails closed.

5 What we measured

Every number in this document was retrieved from the network's own public interfaces on the dates shown, and every one is reproducible by running the corresponding script in the public repository. This is the credibility of the document, so it is stated compactly and without interpretation.

Quantity	Value	Retrieved
Nodes, all tiers	6,489	2026-09-03
Cumulus / Nimbus / Stratus	3,247 / 1,615 / 1,627	2026-09-03
Collateral locked	88,514,500 FLUX	2026-09-03
Applications deployed	1,195	2026-09-03
on specification v8	873 (73.05 %)	2026-09-03
scientific-simulation deployments	27	2026-09-03
Issued main-chain supply	429,466,823.97 FLUX	height 2,912,015
Shielded pools, Sprout + Sapling	96,479.94 FLUX	height 2,917,115
Parallel-asset liability, all ten chains	223,027,781.21 FLUX	2026-09-03/04
exposed to the redemption window	6,226,900.80 FLUX (2.79 %)	2026-09-03/04
Proof of Node activation	block 2,020,000	2025-10-25

Two of these were harder to obtain than they look, and are worth a sentence each because they correct impressions the raw figures give.

Parallel-asset liability is not total supply. Each parallel asset was minted at its full nominal amount — 440,000,000 on most chains — and the issuer still holds nearly all of it. The figure that matters for consolidation is what left the issuer's hands, which required the issuer's own address registry to compute. On Solana, for instance, nominal supply is sixty million and the outstanding balance is 373,067.20.

Two chains needed a different method. Kadena's token contract exposes no total-supply function; supply was recovered by enumerating every account across its twenty chains and summing. Tron required the block explorer's API rather than the node's. Both are scripted.

What this does not do yet. The table states what is public. It does not state utilisation, which no public interface reports; per-owner concentration beyond what the chain exposes, which needs beneficial-ownership data; or real token rates for inference, which are scheduled to be measured in Q4 2026. Where a number is absent here, it is because it could not be established rather than because it was not looked for.

6 What does not work yet

A document that only said the first five sections would be a pitch. This one is the reason to believe the rest. Each row is a finding from the full paper, stated once and without softening; the last column is what, if anything, is being done about it.

	The limit	Why it matters, and the response
Hosting pays nothing extra	PNR pays every eligible node in a tier the same share, regardless of what it runs. The marginal reward for hosting an application is exactly zero.	Free-riding is individually rational; what compels hosting is the eligibility gate, and that gate is currently bound to neither the operator nor the machine. This is deliberate — paying by placement would make the claim race a race for money — and the answer is attested execution, which is why that work is a precondition rather than a follow-on.
Four parties reach the removal threshold	At the measured collateral distribution, four owner identities together hold 25 % of voting weight. The moderation quorum is not censorship-resistant today.	Accepted knowingly, with growth as the expected remedy. The paper notes the tension: the network's answer to weak demand is contraction, and contraction concentrates. Censorship resistance is, in effect, a function of demand for the cloud.
No chain-level privacy	The shielded pool is being retired. Every payment will be publicly linkable.	A deliberate trade for a smaller, auditable chain. Payment privacy becomes an open problem, and the paper says so rather than claiming otherwise.
One discovery endpoint	The service that tells the network which applications exist and where they run is a single Foundation-operated API, with no on-chain or peer-to-peer fallback.	If it goes down, nothing new propagates; the last-known configuration keeps serving. A real control-plane single point of failure, stated in the full paper's centralisation inventory.
Four keys can produce blocks	An emergency path lets a 2-of-4 key set produce blocks that bypass the node-eligibility check, with no time or stall-detection gating on when it may be used.	Intended for recovery from a stalled chain, and the set is planned to move to 3-of-4 with fresh keys. But the gating is absent at either size: nothing in consensus limits *when* the path fires, only *who* can fire it. The full paper lists it in the centralisation inventory alongside the discovery endpoint.
The bridge is administered	Mint, release and upgrades are 3-of-4 among four people from one organisation, and the contracts are upgradable.	Explicitly a bring-up safety net, with immutability the stated destination once a third-party audit and a proven operating record exist. Until the upgrade path is renounced on chain, the paper describes the bridge as administered, not trust-minimized.
Accelerators attach whole	A GPU is passed through as an entire device. There is no subdivision, and a request under contention is silently dropped rather than refused.	This is the one thing the small-model thesis of Section 4 needs and the current allocation cannot do. Planned with the cloud/edge merge; not shipped.

None of these is hidden in the full paper, and none should be hidden here. A reader deciding whether to run a node, deploy an application, or build on this platform is better served by knowing them than by discovering them.

7 Where this goes

7.1 The dates

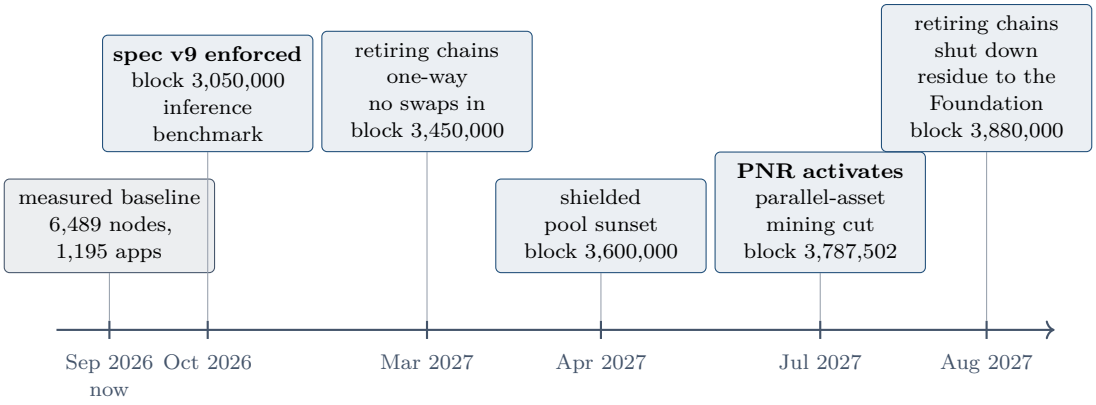

Figure 4: The dated schedule. Heights are consensus facts and calendar dates assume the thirty-second block target holds. Everything to the right of "now" is planned.

Three things are true about this schedule and worth stating together. The heights are fixed and public, so anyone can check whether they were met. The order is deliberate: the specification change lands roughly eight months before the economic change that depends on it, so the fields PNR needs have time to prove themselves. And the whole sequence is falsifiable — the full paper names, for each milestone, the public observation that would show it had not happened.

7.2 How to participate

Run a node. Lock the collateral for a tier, run the node software on hardware that clears that tier's benchmark, and the chain does the rest. Income today is issuance by rotation; from July 2027 it is issuance plus a share of application revenue. What you cannot do is earn more by hosting more — see Section 6 before deciding that is what you want.

Deploy an application. Write a specification, sign it, pay for it. There is no account and no approval step. The application runs on several nodes until the paid period ends, and renewing it is your act. If you want subscriptions and cards, a third-party service offers them on top; the protocol does not.

Build for machines. If your tenant is a program — an agent, a pipeline, a system that provisions and releases capacity on its own — Flux is being built for exactly that caller. Give it a key, fund the key with what you are willing to lose, and it needs nothing else from anyone.

Read the source. The chain, the node software and the tooling are public repositories, and the full paper cites them by file and line for every claim about behaviour. Where it cites a private repository, it says so.

Read further

This document is a door into a longer one. *Flux v9: A Decentralized Cloud for Autonomous Compute* runs to over four hundred pages, cites source code by file and line for every claim about deployed behaviour, proves what can be proved, and marks every undecided element as such. Nothing here goes beyond it; everything here is compressed from it. If a claim in this document seems too strong, too weak, or too neat, the section below is where to check.

If you want…	…read, in the full paper
the system model and the adversary it is defended against	§2, System model
the network as measured, with methodology	§3, Flux as deployed today; §25, Evaluation
how Proof of Node works, and why fork choice is a block-count race	§4, Consensus
the full emission schedule and supply function	§5, Monetary policy; Appendix C
the parallel-asset measurement, chain by chain	§6, Parallel assets
PNR specified: the pool, the drip, the invariants, and why hosting pays nothing extra	§7, Progressive Node Rewards
how placement actually works, with the ninety-second rule	§9, FluxOS
the v9 specification and what consumes it	§10–§11
the boot chain, and what a tenant can and cannot verify	§14, Attested execution
the small-model argument, with the capacity estimates and their caveats	§15, Accelerated compute
what an agent can hold, sign and be bounded by	§17–§18
the bridge construction, its invariants, and the quorum's limits	§22, Trust-minimized bridging
the moderation quorum and the concentration measurement	§23, Governance
the shielded pool, its retirement, and the post-quantum finding	§20, §8
everything that does not hold, in one place	§28, Limitations

The full paper, its evidence corpus, and every script that produced a number in either document are in the public repository.